Bits y E/S mapeada en memoria

Encender un LED es **escribir un bit en una dirección de memoria**. El pin pasa a **5 V** (AVR) o **3,3 V** (ESP32).

Hardware · Software · código · Regla · decisión · Error · ojo · ◇ decisión

1 Binario → decimal → hex

ejemplo: el byte 1100 1000

1 Un byte son 8 bits

Se numeran de **derecha a izquierda**. Cada bit tiene un peso.

bit 7	bit 6	bit 5	bit 4	bit 3	bit 2	bit 1	bit 0
1	1	0	0	1	0	0	0
128	64	32	16	8	4	2	1

2 Decimal: sumo los pesos de los 1

```
128 + 64 + 8 = 200
```

3 Hex: agrupo de 4 en 4

1 dígito hex = 4 bits. En C: `0x` es hex y `0b` es binario.

```
1100 | 1000 → C | 8 → 0xC8
```

4 De decimal a binario: resto los pesos

De mayor a menor.

```
200 − 128 − 64 − 8 = 0 → 1100 1000
```

UN DÍGITO HEX = 4 BITS

0000 0	0001 1	0010 2	0011 3	0100 4	0101 5	0110 6	0111 7
1000 8	1001 9	1010 A	1011 B	1100 C	1101 D	1110 E	1111 F

2 Compuertas y operadores en C

A	B	AND &	OR |	XOR ^	NOT A ~
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

CADA COMPUERTA HACE ESTO

AND
1 solo si ambos son 1.

OR
1 si alguno es 1.

XOR
1 si son distintos.

NOT
Invierte el bit.

EJEMPLO: 0XCA & 0X0F, COLUMNA POR COLUMNA

```
  0xCA = 1100 1010
& 0x0F = 0000 1111
  ---------------
  0x0A = 0000 1010
```

3 Qué hacer con el bit n

una máscara toca solo los bits que quiero

n

Máscara
Byte con 1 solo en los bits a tocar. `1 << 5` = `0010 0000` = `0x20`.

Encender
OR con 1 pone 1.

```
REG |= (1 << n);
```

Apagar
AND con 0 pone 0.

```
REG &= ~(1 << n);
```

Invertir
XOR con 1 cambia el bit.

```
REG ^= (1 << n);
```

Leer
Distinto de 0 si el bit es 1.

```
if (REG & (1 << n))
```

ERROR CLÁSICO

`REG = (1 << 2);` con `=` en vez de `|=` **borra todos los demás bits**. Lo correcto: `REG |= (1 << 2);`

4 DDR → PORT → estado del pin

el bit n de cada registro es el pin n del puerto

1

DDRx: entrada o salida BIT N
Bit en 1 = **salida**.

2

PORTx: el valor BIT N
Salida: el valor que sale. Entrada: **activa la pull-up**.

Flotante
Lee un valor al azar.

```
DDR 0 · PORT 0
```

Pull-up
Suelto = 1, pulsado = 0.

```
DDR 0 · PORT 1
```

Salida 0 V
LED apagado.

```
DDR 1 · PORT 0
```

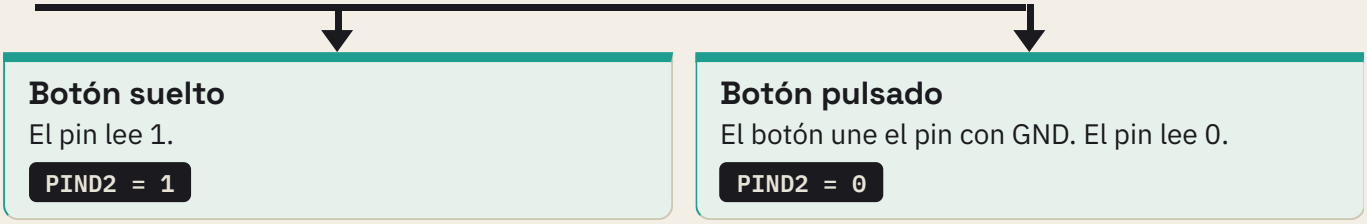
Salida 5 V
LED encendido.

```
DDR 1 · PORT 1
```

PINX: LEER EL PIN

Registro que lee el **voltaje real** del pin. Bit en 1 = pin en alto.

BOTÓN CON PULL-UP (PIND2)



Botón suelto
El pin lee 1.
`PIND2 = 1`

Botón pulsado
El botón une el pin con GND. El pin lee 0.
`PIND2 = 0`

5 Arduino vs. registros (AVR)

pin 13 del Uno = PB5 · pin 2 = PD2

FUNCIONES DE ARDUINO	REGISTROS
`pinMode(13, OUTPUT);`	`DDRB |= (1 << 5);`
`digitalWrite(13, HIGH);`	`PORTB |= (1 << 5);`
`digitalWrite(13, LOW);`	`PORTB &= ~(1 << 5);`
`pinMode(2, INPUT_PULLUP);`	`DDRD &= ~(1 << 2); PORTD |= (1 << 2);`
`int b = digitalRead(2);`	`int b = (PIND & (1 << 2)) != 0;`